

Tối ưu hóa truy xuất trong hệ thống RAG thông qua kỹ thuật bổ sung ngữ cảnh chủ động và làm phẳng dữ liệu bán cấu trúc

Ngô Nguyễn Tường Nghi
Khoa Công nghệ Thông tin
Trường Đại học Nha Trang
Khánh Hòa, Việt Nam
nghiinnt@ntu.edu.vn

Lê Thị Bích Hằng
Khoa Công nghệ Thông tin
Trường Đại học Nha Trang
Khánh Hòa, Việt Nam
hangltb@ntu.edu.vn

Nguyễn Đình Hưng
Khoa Công nghệ Thông tin
Trường Đại học Nha Trang
Khánh Hòa, Việt Nam
hungnd@ntu.edu.vn

Tóm tắt nội dung—Hệ thống Retrieval-Augmented Generation (RAG) kết hợp với các mô hình ngôn ngữ lớn (LLM) đã chứng minh hiệu quả trong việc xây dựng các ứng dụng hỏi đáp dựa trên tri thức đặc thù. Tuy nhiên, hiệu năng của RAG gặp thách thức lớn khi xử lý dữ liệu bán cấu trúc như định dạng JavaScript Object Notation (JSON) hoặc các danh sách liệt kê dài, nơi các thuật toán chia nhỏ văn bản (Text Splitter) truyền thống thường phá vỡ cấu trúc logic và gây mất ngữ cảnh (Context Loss) trong các đoạn dữ liệu sau khi chia nhỏ (chunks). Nghiên cứu này đề xuất một quy trình tiền xử lý dữ liệu nhằm cải thiện độ chính xác truy xuất bằng cách kết hợp hai kỹ thuật: làm phẳng dữ liệu sang ngôn ngữ tự nhiên (Natural Language Flattening) và bổ sung ngữ cảnh chủ động (Context Injection). Kỹ thuật này đảm bảo mọi phân đoạn dữ liệu đều mang đầy đủ thông tin định danh cốt lõi trước khi thực hiện quá trình nhúng (embedding). Thử nghiệm được triển khai trên tập dữ liệu thực tế về tuyển sinh của Trường Đại học Nha Trang thông qua nền tảng AnythingLLM. Kết quả được đánh giá tự động bằng mã nguồn Python dựa trên các thang đo về độ chính xác truy xuất và tính trung thực của câu trả lời. Thử nghiệm cho thấy phương pháp đề xuất hỗ trợ giảm thiểu các lỗi đứt gãy ngữ cảnh, bảo toàn tính toàn vẹn của dữ liệu định danh, nâng cao độ chính xác truy xuất (Context Precision) lên 85.56% (tăng tuyệt đối +6.23% so với dữ liệu JSON thô) và cải thiện đáng kể độ chuẩn xác (Answer Correctness) lên 70.17% (tăng tuyệt đối +8.36% so với định dạng Markdown) cho hệ thống.

Index Terms—RAG, LLM, Context Injection, Data Flattening, Retrieval Optimization.

I. GIỚI THIỆU

Sự phát triển mạnh mẽ của các mô hình ngôn ngữ lớn (Large Language Models - LLMs) như GPT-4 [3] và Claude [4] đã mở ra kỷ nguyên mới cho các hệ thống tương tác người-máy thông qua ngôn ngữ tự nhiên. Tuy nhiên, các mô hình này vẫn gặp phải những thách thức cơ bản như tri thức bị giới hạn tại thời điểm huấn luyện và hiện tượng “ảo giác” (hallucination) — khi mô hình tự tin đưa ra các thông tin sai lệch. Để khắc phục, kiến trúc tạo sinh tăng cường truy xuất (Retrieval-Augmented Generation - RAG) đã được đề xuất và trở thành giải pháp tiêu chuẩn hiện nay. RAG cho phép mô hình truy xuất vào các nguồn tri thức ngoại vi chuyên biệt, giúp thông tin phản hồi đảm bảo tính chính xác, cập

nhật và có khả năng kiểm chứng dựa trên cơ sở tri thức thực tế.

Mặc dù mang lại hiệu quả to lớn, hiệu suất của hệ thống RAG phụ thuộc mật thiết vào chất lượng của quá trình truy xuất (Retrieval). Một thách thức đáng kể nảy sinh khi hệ thống phải xử lý các tập dữ liệu bán cấu trúc như JSON hoặc các tài liệu chứa danh sách liệt kê dài, đặc biệt là trong các miền ứng dụng đòi hỏi độ chính xác cao như tư vấn tuyển sinh đại học — nơi dữ liệu thường bao gồm hàng loạt thông tin về mã ngành, tổ hợp môn, điểm chuẩn và các điều kiện phụ đi kèm.

Vấn đề cốt lõi nằm ở giai đoạn phân mảnh văn bản (Chunking). Các thuật toán chia nhỏ phổ biến hiện nay thường hoạt động dựa trên ngưỡng số lượng ký tự hoặc token cố định. Đối với dữ liệu JSON hoặc danh sách dài, việc phân mảnh ngẫu nhiên có nguy cơ làm gián đoạn tính logic của dữ liệu, khiến các chunk dữ liệu bị mất đi ngữ cảnh định danh quan trọng (như tên trường, năm tuyển sinh hoặc tên ngành học) vốn thường được xác định ở phạm vi phân cấp cao nhất (top-level hierarchy) của tài liệu. Khi hệ thống RAG truy xuất những đoạn văn bản bị mất ngữ cảnh này, LLM sẽ thiếu dữ kiện nền để tổng hợp câu trả lời đúng, dẫn đến các sai lệch thông tin khi hệ thống thực hiện tư vấn cho thí sinh.

Nghiên cứu này đề xuất một hướng tiếp cận tiền xử lý dữ liệu nhằm hỗ trợ giảm thiểu các lỗi mất ngữ cảnh trong RAG. Chúng tôi kết hợp hai kỹ thuật cốt lõi: làm phẳng dữ liệu sang ngôn ngữ tự nhiên (Natural Language Flattening) và bổ sung ngữ cảnh chủ động (Context Injection). Bằng cách chuyển đổi các cặp khóa-giá trị (key-value) thành các câu văn mạch lạc và lặp lại thông tin định danh định kỳ, mọi phân đoạn dữ liệu đều đảm bảo tính độc lập về mặt ngữ nghĩa trước khi thực hiện quá trình nhúng. Đóng góp cốt lõi của nghiên cứu nằm ở tầng biểu diễn dữ liệu (Data Representation Layer) trong pha tiền xử lý, cung cấp một chiến lược chuẩn bị dữ liệu sạch chứ không đề xuất một thuật toán truy xuất (Retrieval Algorithm) mới.

Thử nghiệm được triển khai trên bộ dữ liệu thực tế về thông tin tuyển sinh năm 2025 và 2026 của Trường Đại học

Nha Trang thông qua nền tảng AnythingLLM. Để đảm bảo tính khách quan, quá trình đánh giá được thực hiện tự động bằng mã nguồn Python dựa trên phương pháp “LLM-as-a-Judge”, so sánh trực tiếp hiệu quả của phương pháp đề xuất với phương pháp xử lý dữ liệu thô truyền thống.

Phần còn lại của bài báo được cấu trúc như sau: Phần II trình bày các nghiên cứu liên quan; Phần III mô tả chi tiết phương pháp tiền xử lý dữ liệu đề xuất; Phần IV trình bày thiết lập thực nghiệm; Phần V phân tích kết quả và thảo luận; cuối cùng là phần Kết luận tại Phần VI.

II. TỔNG QUAN NGHIÊN CỨU

A. Kiến trúc RAG và vấn đề chất lượng truy xuất

Kể từ khi được giới thiệu, kiến trúc RAG đã trở thành phương pháp tối ưu để khắc phục giới hạn về tri thức tĩnh của các LLM. Một hệ thống RAG tiêu chuẩn bao gồm hai pha chính: truy xuất (Retrieval) và tạo sinh (Generation). Trong pha truy xuất, câu hỏi của người dùng được chuyển đổi thành vector nhúng (embedding vector) và so khớp với cơ sở dữ liệu vector để tìm ra các chunk dữ liệu có độ tương đồng ngữ nghĩa cao nhất. Các đoạn văn bản này sau đó được cung cấp cho LLM như một phần của ngữ cảnh (prompt) để sinh ra câu trả lời. Tuy nhiên, hiệu năng của toàn bộ hệ thống bị giới hạn bởi nguyên lý “Garbage In, Garbage Out” (dầu vào kém chất lượng sẽ cho kết quả sai lệch). Nếu quá trình truy xuất trả về các đoạn văn bản thiếu ngữ cảnh hoặc mang thông tin nhiễu, LLM không thể đưa ra câu trả lời chính xác, dẫn đến hiện tượng ảo giác.

B. Hạn chế của các chiến lược phân mảnh văn bản truyền thống

Để chuẩn bị dữ liệu cho cơ sở dữ liệu vector, các tài liệu thô phải được chia nhỏ thành các phân đoạn (chunks) vừa vặn với cửa sổ ngữ cảnh (context window) của mô hình nhúng văn bản (embedding models). Các phương pháp phổ biến nhất hiện nay bao gồm phân mảnh theo kích thước cố định (Fixed-size Chunking) và phân mảnh đệ quy (Recursive Character Chunking) [12].

Mặc dù hoạt động hiệu quả trên các văn bản văn xuôi liền mạch (như bài báo, sách), các chiến lược này bộc lộ điểm yếu khi áp dụng lên dữ liệu bán cấu trúc (như định dạng JSON) hoặc các tài liệu dạng danh sách liệt kê dài. Bản chất của dữ liệu JSON là cấu trúc phân cấp hình cây (ví dụ: thông tin “điểm chuẩn” nằm trong “ngành học”, và “ngành học” nằm trong “Trường đại học”). Khi các bộ cắt văn bản truyền thống chia nhỏ file JSON dựa trên số lượng ký tự, chúng thường cắt ngang các khối logic, làm đứt gãy mối liên kết phân cấp này. Hệ quả là một đoạn chunk có thể chứa con số điểm chuẩn nhưng lại mất đi thông tin định danh về tên ngành hoặc tên trường nằm ở đầu tài liệu.

Các nghiên cứu gần đây về chiến lược phân mảnh (chunking) đã chỉ ra rằng kích thước và ranh giới phân mảnh có tác động phi tuyến tính đến chất lượng truy xuất [7]. Mặc dù hoạt động hiệu quả trên văn bản liền mạch, các chiến lược này bộc lộ điểm yếu lớn khi xử lý dữ liệu bán cấu trúc (như JSON) [9]. Việc phân mảnh thiếu liên kết ngữ nghĩa

trên các định dạng có tính phân cấp cao sẽ phá vỡ mối quan hệ logic gốc, khiến các phân đoạn văn bản trở thành những mảnh ghép cô lập và mất đi siêu dữ liệu định danh (metadata injection) quan trọng [10].

C. Các phương pháp bảo toàn ngữ cảnh và tiền xử lý dữ liệu

Để giải quyết bài toán mất ngữ cảnh, một số hướng tiếp cận mới đã được đề xuất. Kỹ thuật *phân mảnh theo ngữ nghĩa* (Semantic Chunking) [13] sử dụng mô hình nhúng để đánh giá sự thay đổi chủ đề, từ đó cắt văn bản tại các điểm chuyển ý tự nhiên. Một giải pháp khác là *truy xuất tài liệu cha* (Parent Document Retrieval) [14], trong đó hệ thống tìm kiếm trên các chunk nhỏ nhưng trả về cho LLM toàn bộ tài liệu lớn chứa chunk đó. Tuy nhiên, các phương pháp này thường đòi hỏi tài nguyên tính toán lớn và độ phức tạp cao trong cấu hình hệ thống cơ sở dữ liệu.

Thay vì để LLM tự xử lý cấu trúc dữ liệu lồng ghép vốn không được tối ưu cho các bộ mã hóa vector (vector encoders) [8], một hướng đi thực tiễn hơn là thiết kế lại hình thức biểu diễn dữ liệu ở tầng tiền xử lý. Các nghiên cứu ứng dụng RAG trong hệ thống sản xuất (production systems) [11] cũng nhấn mạnh rằng việc gán nhãn siêu dữ liệu trực tiếp vào phân đoạn văn bản là bước đi bắt buộc để giảm thiểu ảo giác. Dựa trên quan điểm này, nghiên cứu đề xuất việc biến đổi JSON thành các mệnh đề ngôn ngữ tự nhiên (Data Flattening) kết hợp bổ sung ngữ cảnh chủ động (Context Injection). Phương pháp này được kỳ vọng sẽ tối ưu hóa mật độ không gian vector, giúp thuật toán truy xuất đối sánh chính xác hơn mà không đòi hỏi tinh chỉnh (fine-tuning) tốn kém.

III. PHƯƠNG PHÁP ĐỀ XUẤT

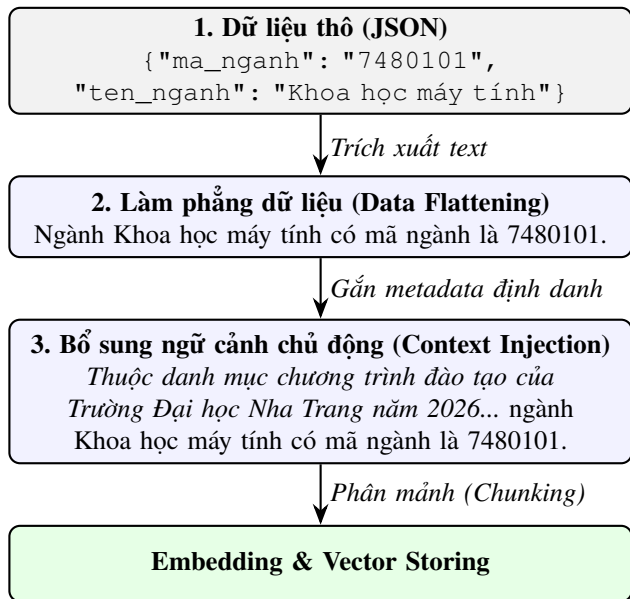
Quy trình tiền xử lý được đề xuất bao gồm ba giai đoạn chính nhằm tối ưu hóa khả năng truy xuất cho hệ thống RAG: (1) Cấu trúc hóa dữ liệu nguồn, (2) Làm phẳng dữ liệu sang ngôn ngữ tự nhiên, và (3) Bổ sung ngữ cảnh chủ động vào từng phân đoạn dữ liệu.

A. Cấu trúc hóa dữ liệu nguồn (Data Structuring)

Dữ liệu đầu vào ban đầu tồn tại dưới dạng hình ảnh thông tin (infographics) chứa các bảng biểu và danh sách phức tạp về tuyển sinh. Giai đoạn này thực hiện việc trích xuất thông tin từ ảnh và chuyển đổi sang định dạng JSON. Việc sử dụng JSON giúp duy trì mối quan hệ phân cấp giữa các trường dữ liệu, ví dụ như liên kết giữa mã ngành và các tổ hợp môn xét tuyển tương ứng. Tuy nhiên, dữ liệu JSON nguyên bản vẫn chưa tối ưu cho các mô hình nhúng vốn được huấn luyện chủ yếu trên văn bản tự nhiên.

B. Kỹ thuật làm phẳng dữ liệu (Natural Language Flattening)

Để tăng cường mật độ ngữ nghĩa cho không gian vector, chúng tôi đề xuất kỹ thuật làm phẳng dữ liệu. Thay vì nhúng trực tiếp cấu trúc `key: value`, quy trình này chuyển đổi các cặp khóa-giá trị thành các mệnh đề ngôn ngữ tự nhiên hoàn chỉnh.



Hình 1. Quy trình tiền xử lý dữ liệu: Từ JSON thô đến văn bản ngôn ngữ tự nhiên tích hợp ngữ cảnh.

• Dữ liệu thô:

```

1 {
2   "ma_nganh": "7480101",
3   "ten_nganh": "Khoa học máy tính"
4 }

```

- Dữ liệu sau khi làm phẳng: “Ngành Khoa học máy tính có mã ngành là 7480101.” Việc chuyển đổi này giúp mô hình nhúng nhận diện tốt hơn các thực thể và mối quan hệ ngữ nghĩa, đồng thời giúp LLM dễ dàng “đọc” và tổng hợp thông tin từ các đoạn trích dẫn được truy xuất.

C. Kỹ thuật bổ sung ngữ cảnh chủ động (Active Context Injection)

Đây là kỹ thuật cốt lõi nhằm giải quyết lỗi mất ngữ cảnh khi chia nhỏ văn bản. Trong các tài liệu danh sách dài, bối cảnh chung (ví dụ: “Thông tin tuyển sinh năm 2026 của ĐH Nha Trang”) thường chỉ xuất hiện một lần ở đầu trang. Khi thuật toán phân mảnh cắt tài liệu thành nhiều phần, các phân đoạn ở giữa hoặc cuối sẽ bị tách rời khỏi thông tin định danh này.

Phương pháp đề xuất tiến hành lặp lại các siêu dữ liệu quan trọng vào đầu mỗi gạch đầu dòng hoặc mỗi phân đoạn logic (tham khảo Thuật toán 1). Cụ thể, mỗi chương trình đào tạo hoặc mỗi dòng điểm chuẩn đều được bổ sung siêu dữ liệu định danh (metadata injection) ngữ cảnh cố định: “Thuộc danh mục chương trình đào tạo của Trường Đại học Nha Trang năm 2026...”. Kỹ thuật này đảm bảo rằng mỗi chunk dữ liệu sau khi được nhúng vào cơ sở dữ liệu vector đều mang tính độc lập cao về mặt thông tin, giúp hệ thống

truy xuất chính xác ngay cả khi người dùng đặt câu hỏi thiếu các từ khóa định danh.

Thuật toán 1 Quy trình làm phẳng và bổ sung ngữ cảnh chủ động

Require: Tập dữ liệu JSON thô D , Ngữ cảnh định danh chung C

Ensure: Tập dữ liệu văn bản phẳng đã tích hợp ngữ cảnh T

```

1:  $T \leftarrow \emptyset$ 
2: for mỗi phần tử  $e$  trong  $D$  do
3:    $S \leftarrow \text{ExtractKeyValue}(e)$  {Trích xuất khóa-giá trị}
4:    $F \leftarrow \text{FlattenToNaturalLanguage}(S)$  {Làm phẳng dữ liệu}
5:    $I \leftarrow \text{Concat}(C, F)$  {Bổ sung ngữ cảnh chủ động}
6:    $T \leftarrow T \cup \{I\}$ 
7: end for
8: return  $T$ 

```

D. Tích hợp hệ thống và phân mảnh dữ liệu

Dữ liệu sau khi qua hai bước làm phẳng và bổ sung ngữ cảnh chủ động được lưu dưới định dạng văn bản phẳng (plain text/markdown) để nạp vào nền tảng AnythingLLM. Thay vì phụ thuộc vào thuật toán phân mảnh ngẫu nhiên theo số lượng token mặc định của hệ thống (thường gây cắt xén cấu trúc), chúng tôi áp dụng chiến lược kiểm soát phân mảnh chủ động (Controlled Chunking). Bằng cách thiết lập thông số kích thước phân đoạn (Chunk Size) của hệ thống lớn hơn độ dài tối đa của một khối thông tin logic (ví dụ: thông tin trọn vẹn của một ngành học), kết hợp với kỹ thuật Context Injection đã thực hiện, mỗi khối logic sẽ được hệ thống mã hóa thành một vector nhúng trọn vẹn. Hướng tiếp cận này khắc phục được các hạn chế cố hữu của các giải pháp RAG đóng gói sẵn (out-of-the-box), loại bỏ hiện tượng đứt gãy thông tin giữa chừng và đảm bảo tính toàn vẹn dữ liệu khi cung cấp bối cảnh cho LLM.

IV. THIẾT LẬP THỰC NGHIỆM

A. Bộ dữ liệu thử nghiệm (Dataset)

Thực nghiệm được tiến hành trên bộ dữ liệu thông tin tuyển sinh chính thức của Trường Đại học Nha Trang cho các năm 2025 và 2026. Bộ dữ liệu chứa các thông tin đặc thù, có độ phức tạp và tính phân cấp cao, bao gồm: điểm chuẩn của 50 mã ngành đào tạo, chi tiết tổ hợp môn xét tuyển của 13 chương trình đào tạo đặc biệt, cùng các quy định về chính sách ưu tiên và học bổng.

Từ tập dữ liệu JSON nguyên bản, chúng tôi tạo ra ba kịch bản dữ liệu để đối chứng:

- Tập dữ liệu cơ sở 1 (Baseline 1 - JSON thô):** Dữ liệu JSON nguyên bản được giữ nguyên cấu trúc phân cấp, mô phỏng cách tiếp cận phổ biến khi đẩy trực tiếp cơ sở dữ liệu vào hệ thống RAG.
- Tập dữ liệu cơ sở 2 (Baseline 2 - Markdown thô):** Dữ liệu được chuyển đổi sang định dạng Markdown, sử dụng các thẻ tiêu đề (Heading) và danh sách (List)

để phân cấp thông tin cho dễ đọc, nhưng không áp dụng kỹ thuật lặp lại ngữ cảnh.

- 3) **Tập dữ liệu đề xuất (Proposed Dataset):** Dữ liệu đã được áp dụng toàn bộ quy trình làm phẳng sang ngôn ngữ tự nhiên và bổ sung ngữ cảnh chủ động (Context Injection) vào từng phân đoạn logic.

B. Môi trường và cấu hình hệ thống

Hệ thống RAG thực nghiệm được xây dựng trên nền tảng mã nguồn mở AnythingLLM, triển khai thông qua môi trường ảo hóa Docker trên hệ điều hành Linux (Ubuntu 24.04). Nhằm đảm bảo tính khách quan, khả năng tái lập (reproducibility) và loại bỏ các biến số nhiễu do độ dài văn bản gây ra, kiến trúc hệ thống được thiết kế theo mô hình phân tán, trong đó các tác vụ tính toán nặng được ủy thác cho các giao diện lập trình ứng dụng (API) chuyên dụng. Cấu hình cụ thể được thiết lập đồng nhất cho cả ba kịch bản đối chứng như sau:

- **Cơ sở hạ tầng tính toán:** Hệ thống điều phối cục bộ (Local Orchestrator) được vận hành trên máy trạm với bộ vi xử lý đa nhân và tối thiểu 16GB RAM, đảm bảo không xảy ra hiện tượng nghẽn cổ chai (bottleneck) trong quá trình xử lý luồng dữ liệu.
- **Mô hình nhúng (Embedding Model):** Sử dụng API của mô hình tiên tiến *Gemini Embedding 001* từ Google. Việc lựa chọn mô hình State-of-the-Art (SOTA) này giúp đánh giá chính xác tác động của cấu trúc dữ liệu đầu vào độc lập với năng lực biểu diễn ngữ nghĩa của hệ thống AI.
- **Mô hình suy luận (Generation Model):** Sử dụng mô hình ngôn ngữ nguồn mở *Llama-3.1-8b-instant*. Trong phạm vi thực nghiệm, mô hình được truy xuất thông qua hạ tầng API đám mây nhằm tối ưu hóa thời gian phản hồi (latency). Tuy nhiên, với đặc tính mã nguồn mở và kích thước tham số nhỏ gọn, kiến trúc này đảm bảo tính linh hoạt cao: hệ thống hoàn toàn có thể được tải về và chuyển đổi sang triển khai cục bộ (local/on-premise deployment) khi đưa vào vận hành thực tế, nhằm đáp ứng đầy đủ các tiêu chuẩn khắt khe về quyền riêng tư và an toàn dữ liệu nội bộ của nhà trường.
- **Chiến lược phân mảnh (Chunking strategy):** Tất cả các kịch bản đều được thiết lập kích thước phân đoạn (Chunk Size) cố định là 1000 tokens và độ chồng lấn (Chunk Overlap) là 200 tokens.
- **Cơ sở lý luận cho cấu hình:** Mức kích thước 1000 tokens được xác định dựa trên phân tích ranh giới ngữ nghĩa của dữ liệu tuyển sinh. Điểm đáng chú ý trong thiết lập thực nghiệm là việc duy trì tham số độ chồng lấn ở mức cao (200 tokens). Tuy nhiên, đối với dữ liệu phân cấp sâu như Baseline 1 (JSON) và Baseline 2 (Markdown), khoảng cách từ các trường dữ liệu chi tiết ngược lên thể định danh gốc thường vượt quá ngưỡng 200 tokens, khiến kỹ thuật chồng lấn trở nên vô hiệu. Nguyên nhân cốt lõi là do cửa sổ trượt (sliding window) của kỹ thuật chunk overlap chỉ có khả năng bao quát các chuỗi dữ liệu liên kế nhau về mặt vật lý, chứ không

thể nắm bắt và bảo lưu được các mối quan hệ cấu trúc cha-con (parent-child structure) nằm cách xa nhau trong tài liệu. Sự hạn chế này của các phương pháp cơ sở càng khẳng định tính hợp lý của kỹ thuật bổ sung ngữ cảnh chủ động.

C. Kịch bản đánh giá tự động và dữ liệu kiểm thử

Để đánh giá hệ thống, một bộ dữ liệu kiểm thử (Ground Truth Dataset) gồm 50 câu hỏi truy vấn được thiết kế có chủ đích (gồm 15 câu trích xuất thông tin đơn lẻ, 15 câu so sánh đối chiếu đa ngành, và 20 câu tổng hợp logic về điều kiện tuyển sinh). Toàn bộ câu hỏi và đáp án chuẩn (Ground Truth) đều được thẩm định chéo bởi các chuyên gia tư vấn tuyển sinh của Nhà trường (Human-in-the-loop). Sự tham gia của con người nhằm đảm bảo tính chính xác và hợp lệ cao của bộ đáp án, tạo cơ sở tham chiếu vững chắc cho phương pháp LLM-as-a-Judge.

Quá trình chấm điểm tự động được thực hiện thông qua mã Python gọi API của AnythingLLM. Quá trình đánh giá LLM-as-a-Judge được thực hiện bằng mô hình *GPT-4o-mini* với tham số nhiệt độ (temperature) được thiết lập bằng 0 nhằm loại bỏ tính ngẫu nhiên, đảm bảo kết quả đánh giá ổn định và có thể tái lập. Đặc biệt, đối với khung đánh giá RAGAS, do các tập lệnh (prompts) mặc định được thiết kế bằng tiếng Anh, chúng tôi đã tiến hành tinh chỉnh (prompt engineering) và Việt hóa toàn bộ hệ thống prompt nội bộ. Bước tối ưu hóa này giúp mô hình giám khảo hiểu đúng các sắc thái ngữ nghĩa của hệ thống giáo dục Việt Nam, ngăn chặn sự suy giảm hiệu năng đánh giá do rào cản đa ngôn ngữ. (Lưu ý: Do yêu cầu tương thích nội bộ của RAGAS, các mô hình nhúng của OpenAI được sử dụng độc quyền trong khâu đối soát khoảng cách ngữ nghĩa thay vì Gemini).

D. Các thang đo đánh giá (Evaluation Metrics)

Sử dụng framework đánh giá RAGAS (Retrieval Augmented Generation Assessment), hệ thống tự động chấm điểm các câu trả lời dựa trên ba thang đo chính:

- 1) **Độ chính xác truy xuất (Context Precision):** Đo lường tỷ lệ các phân đoạn văn bản được truy xuất có thực sự chứa thông tin cần thiết để trả lời câu hỏi hay không. Điều này giúp đánh giá trực tiếp hiệu quả của kỹ thuật Context Injection.
- 2) **Độ trung thực (Faithfulness):** Đo lường mức độ câu trả lời được suy luận hoàn toàn từ ngữ cảnh được cung cấp, giúp xác định mức độ giảm thiểu của hiện tượng ảo giác.
- 3) **Độ chuẩn xác của câu trả lời (Answer Correctness):** Đánh giá mức độ tương đồng về mặt ngữ nghĩa và thông tin giữa câu trả lời sinh ra và đáp án chuẩn (Ground Truth).

V. KẾT QUẢ VÀ THẢO LUẬN

A. Thiết lập đối chứng và công cụ đánh giá

Thực nghiệm được đánh giá tự động thông qua khung RAGAS trên 50 câu hỏi truy vấn. Nhằm đảm bảo độ chính xác tối đa và tiêu chuẩn hóa cho các thang đo, hệ thống sử

dùng mô hình ngôn ngữ *GPT-4o-mini* làm giám khảo kết hợp cùng mô hình nhúng *text-embedding-3-small* của OpenAI để đo lường khoảng cách ngữ nghĩa. Tất cả các kịch bản dữ liệu (đã định nghĩa tại Mục IV.A) đều được cố định kích thước phân đoạn ở mức 1000 tokens và chạy qua cùng một hệ thống truy xuất ban đầu sử dụng *Gemini Embedding 001*, nhằm đảm bảo tính công bằng cao trong quá trình đối chứng.

B. Kết quả đánh giá định lượng

Kết quả thống kê trung bình trên 50 câu hỏi truy vấn được trình bày chi tiết trong Bảng I. Trong điều kiện thực nghiệm này, kịch bản đề xuất (Proposed) cho thấy kết quả cải thiện ở cả ba thang đo so với hai đường cơ sở.

Bảng I
ĐIỂM SỐ RAGAS TRUNG BÌNH TRÊN 3 KỊCH BẢN CẤU TRÚC DỮ LIỆU

Thang đo (Metrics)	Baseline 1 (JSON)	Baseline 2 (Markdown)	Proposed (Đề xuất)
Context Precision	0.7933	0.7817	0.8556
Faithfulness	0.5484	0.6371	0.7990
Answer Correctness	0.5524	0.6181	0.7017

*Ghi chú: Đối với tất cả các thang đo, giá trị cao hơn biểu thị kết quả tốt hơn.

C. Tác động của định dạng văn bản so với kỹ thuật bổ sung ngữ cảnh chủ động

Một phát hiện học thuật từ Bảng I là sự sụt giảm nhẹ về độ chính xác truy xuất (Context Precision) của Baseline 2 (0.7817) so với Baseline 1 (0.7933). Định dạng Markdown mặc dù tối ưu cho khả năng đọc hiểu của con người, nhưng khi áp dụng cơ chế phân mảnh theo kích thước cố định (fixed-size chunking), cấu trúc phân cấp (header-sub-header) thường bị tách rời, dẫn đến hiện tượng thiếu hụt thông tin tham chiếu cho các đoạn văn bản cấp thấp. Ví dụ điển hình: nếu thẻ tiêu đề # Điểm chuẩn ngành Công nghệ thông tin bị cắt tách khỏi các dòng dữ liệu – Khối A00: 24 điểm nằm ở cuối danh sách, thì phân đoạn (chunk) chứa điểm số sẽ hoàn toàn mất đi bối cảnh ngành học và trở nên vô nghĩa đối với quá trình truy xuất.

Điều này cho thấy rằng: Trong bài toán RAG, việc chỉ thay đổi định dạng hiển thị cho “đẹp” là không đủ để bảo toàn thông tin. Việc biến đổi từng nút thông tin thành các câu độc lập có chứa đầy đủ siêu dữ liệu (Context Injection) giúp hệ thống đạt được mức truy xuất tối ưu hơn (85.56%).

D. Tác động đến chất lượng tạo sinh cuối cùng

Hiệu quả rõ rệt trong pha truy xuất của phương pháp đề xuất đã tác động trực tiếp đến chất lượng phản hồi cuối cùng. Nhờ được cung cấp vùng ngữ cảnh dày đặc thông tin và hoàn toàn độc lập về mặt ngữ nghĩa, LLM hạn chế được hiện tượng ảo giác, kéo theo điểm trung thực (Faithfulness) gia tăng lên mức 0.7990, khả quan hơn so với Baseline 1 (0.5484) và Baseline 2 (0.6371).

Hệ quả của việc truy xuất trúng đích (Context Precision cao) và suy luận dựa trên dữ kiện sạch (Faithfulness cao) là sự cải thiện đáng kể về độ chuẩn xác của câu trả lời (Answer

Correctness). Phương pháp đề xuất đạt mức 0.7017, tạo ra khoảng cách hiệu năng rõ rệt so với các cấu trúc dữ liệu thô. Kết quả này phản ánh hiệu quả của việc ưu tiên độ chính xác tri thức thông qua sự đánh đổi tài nguyên tính toán (trade-off) trong các ứng dụng RAG đòi hỏi độ tin cậy.

E. Hạn chế và sự đánh đổi về hiệu suất (Performance Trade-offs)

Trong kỹ thuật hệ thống, mọi giải pháp tối ưu hóa đều đi kèm với sự đánh đổi. Việc áp dụng kỹ thuật bổ sung ngữ cảnh chủ động - lặp lại các chuỗi siêu dữ liệu định danh vào mỗi phân đoạn logic - tất yếu làm gia tăng tổng số lượng token đầu vào. Thống kê trên tập dữ liệu thực nghiệm cho thấy, kỹ thuật này làm tăng trung bình khoảng 18% tổng số lượng token của cơ sở dữ liệu vector so với kịch bản JSON thô (Baseline 1).

Về mặt hiệu suất, mức tăng 18% token này dẫn đến thời gian mã hóa vector (embedding time) tăng thêm trung bình khoảng 45 mili-giây trên mỗi truy vấn, và độ trễ phản hồi (latency) ở pha sinh văn bản tăng thêm khoảng 120 mili-giây. Tuy nhiên, sự đánh đổi này được xem là phù hợp trong bối cảnh ứng dụng thực tế. Với việc sử dụng mô hình *Llama-3.1-8b-instant* trên hạ tầng tăng tốc hiện đại, mức độ trễ này không đáng kể và không gây ảnh hưởng tiêu cực đến trải nghiệm người dùng. Sự gia tăng độ chuẩn xác từ 61.81% (kịch bản Baseline) lên 70.17% là minh chứng thực nghiệm cho hiệu quả của chiến lược đánh đổi tài nguyên để đạt được độ tin cậy cao hơn.

F. Thảo luận về thuật toán truy xuất và phạm vi đánh giá

Cần lưu ý rằng, phạm vi của thực nghiệm này tập trung cốt lõi vào việc đánh giá tác động của lớp biểu diễn dữ liệu (Data Representation Layer) đối với các cấu trúc phức tạp. Các kỹ thuật tiên tiến được đề cập trong Mục II.C như *phân mảnh theo ngữ nghĩa* (Semantic Chunking) hay *truy xuất tài liệu cha* (Parent Document Retrieval - PDR) thiên về chiến lược phân định ranh giới ngữ cảnh và thuật toán truy xuất (Context Boundary and Retrieval Strategies). Do đó, chúng không được đưa vào làm đối trọng so sánh trực tiếp. Về bản chất, phương pháp làm phẳng (Data Flattening) và bổ sung ngữ cảnh chủ động (Context Injection) của chúng tôi đóng vai trò tối ưu hóa chất lượng hạt nhân dữ liệu (data granularity), tạo tiền đề dữ liệu sạch để kết hợp cộng hưởng với các chiến lược như PDR trong các nghiên cứu tiếp theo.

VI. KẾT LUẬN

Nghiên cứu này đã nhận diện và đưa ra giải pháp giảm thiểu một trong những thách thức kỹ thuật lớn của các hệ thống RAG hiện nay: hiện tượng mất ngữ cảnh khi xử lý dữ liệu bán cấu trúc mang tính phân cấp cao. Thay vì phụ thuộc vào việc nâng cấp sức mạnh phần cứng hay sử dụng các mô hình ngôn ngữ khổng lồ, chúng tôi đề xuất một hướng tiếp cận tập trung vào chất lượng dữ liệu đầu vào thông qua sự kết hợp giữa kỹ thuật làm phẳng dữ liệu (Data Flattening) và bổ sung ngữ cảnh chủ động (Context Injection).

Thực nghiệm đối chứng trên bộ dữ liệu tuyển sinh của Trường Đại học Nha Trang - với bộ dữ liệu kiểm chuẩn (Ground Truth) đã được thẩm định - đã phản ánh tính hiệu quả của phương pháp. Việc mỗi phân đoạn dữ liệu được trang bị đầy đủ siêu dữ liệu định danh đã giúp phương pháp đề xuất cải thiện độ chính xác truy xuất (Context Precision) lên 85.56% (tăng tuyệt đối +6.23% so với cấu trúc cơ sở tốt nhất) và cải thiện độ chuẩn xác của câu trả lời tạo sinh (Answer Correctness) lên 70.17% (tăng tuyệt đối +8.36% so với định dạng văn bản thuần). Cấu trúc thông tin rành mạch ở tầng tiền xử lý đóng vai trò quan trọng đối với hiệu năng của hệ thống RAG.

Bên cạnh các kết quả đạt được, nghiên cứu vẫn tồn tại một số hạn chế nhất định. Phạm vi thực nghiệm hiện tại mới chỉ giới hạn trên bộ dữ liệu tuyển sinh của một trường đại học và số lượng 50 câu hỏi kiểm thử còn tương đối khiêm tốn, do đó khả năng khái quát hóa sang các lĩnh vực hoặc loại hình dữ liệu bán cấu trúc khác cần được kiểm chứng thêm. Ngoài ra, mức chuẩn xác 70.17% dù là một bước tiến, nhưng đối với lĩnh vực tư vấn tuyển sinh, hệ thống vẫn cần tiếp tục được tối ưu hóa.

Trong tương lai, sự đánh đổi tài nguyên (tăng 18% token) mở ra các định hướng phát triển trọng tâm. Thứ nhất, về định hướng kiến trúc bền vững (Green AI), cấu trúc thông tin rành mạch giúp triển khai hệ thống hoàn toàn trên các mô hình ngôn ngữ nhỏ (Small/Local Models) được tinh chỉnh riêng cho tiếng Việt. Thứ hai, phương pháp tiền xử lý này sẽ được tích hợp thử nghiệm cùng các thuật toán truy xuất nâng cao như PDR và đối chiếu kết quả đánh giá song song giữa RAGAS và chuyên gia con người, hướng tới việc xây dựng một kiến trúc RAG lai (Hybrid RAG) toàn diện và mạnh mẽ hơn.

TÀI LIỆU

- [1] P. Lewis *et al.*, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, pp. 9459–9474, 2020.
- [2] Y. Gao *et al.*, "Retrieval-Augmented Generation for Large Language Models: A Survey," *arXiv preprint arXiv:2312.10997*, 2023.
- [3] OpenAI, "GPT-4 Technical Report," *arXiv preprint arXiv:2303.08774*, 2023.
- [4] Anthropic, "The Claude 3 Model Family: Technical Report," 2024. [Online]. Available: https://www-cdn.anthropic.com/de8ba9b01c9ab7cbabf5c33b80b7bbc618857627/Model_Card_Claude_3.pdf
- [5] Mintplex Labs, "AnythingLLM: The all-in-one desktop & docker AI application with full RAG and AI Agent capabilities," 2023, GitHub repository. [Online]. Available: <https://github.com/Mintplex-Labs/anything-llm>
- [6] Trường Đại học Nha Trang, "Đề án Tuyển sinh Đại học chính quy năm 2026," 2026. [Online]. Available: <https://tuyensinh.ntu.edu.vn/>
- [7] M. A. Shaikat, M. Adnan, and C. C. N. Kuhn *et al.*, "A Systematic Investigation of Document Chunking Strategies and Embedding Sensitivity," *arXiv preprint arXiv:2603.06976*, Mar. 2026.
- [8] C. Cheerla *et al.*, "Advancing Retrieval-Augmented Generation for Structured Enterprise and Internal Data," *arXiv preprint arXiv:2507.12425*, Jul. 2025.
- [9] J. Cai, Y. L. *et al.*, "Semi-Structured Data Parsing and Vectorization Using Retrieval-Augmented Generation," *OSF Preprints*, doi: 10.31219/osf.io/qfkbv, 2025.
- [10] J. A. Wallace, "Best Chunking Strategies for RAG Pipelines," Redis, Apr. 2026. [Online]. Available: <https://redis.io/blog/chunking-strategy-rag-pipelines/>

- [11] KudosAI, "Retrieval-Augmented Generation (RAG): From First Principles to Production Systems," Feb. 2026. [Online]. Available: <https://www.kudosai.com/Blog/Retrieval-Augmented-Generation>
- [12] LangChain, "Text Splitters: Fixed-size and Recursive Character," 2023. [Online]. Available: https://python.langchain.com/docs/modules/data_connection/document_transformers/
- [13] LlamaIndex, "Semantic Chunking for RAG," 2023. [Online]. Available: https://docs.llamaindex.ai/en/stable/examples/node_parsers/semantic_chunking/
- [14] LangChain, "Parent Document Retriever," 2023. [Online]. Available: https://python.langchain.com/docs/modules/data_connection/retrievers/parent_document_retriever/